

Python Bridge – Addon-Dokumentation

Version 0.2.1 | Godot 4.2+ (getestet 4.7.x) + Python 3.8+ | WebSocket-basierte Python-Prozess-Instanzen
mit Task-Manager

Inhalt

1. Übersicht
2. Architektur
3. Installation in einem neuen Projekt
4. GDScript-API-Referenz
5. Task Manager & Scheduler
6. Datentransport & Typen
7. Python-Seite
8. Editor-Integration
9. Lebenszyklus, Provisioning & Fehlerbehandlung
10. Troubleshooting

1. Übersicht

Die Python Bridge startet aus Godot heraus einen oder mehrere Python-Subprozesse (je Instanz ein eigener Prozess) und kommuniziert mit ihnen über WebSocket auf `127.0.0.1`. Sie übernimmt automatisch:

- **venv-Verwaltung:** legt pro Projekt ein virtuelles Environment unter `python_bridge/venv/` an (einmalig).
- **Dependency-Management:** installiert `websockets` + frei konfigurierbare Pakete (z. B. `numpy`) per `pip`.
- **Python-Erkennung:** findet eine installierte Python (3.8+) selbst, ohne fest verdrahteten Pfad.
- **Task Manager:** zentrale Warteschlange mit Prioritäten, Timeout, Cancel, Retry und Backpressure.
- **Frame-Synchronisation:** Python-Antworten werden gepuffert und kontrolliert im Haupt-Thread verarbeitet – kein Frame-Stall.
- **Task-Batching:** kurz aufeinanderfolgende Aufrufe werden optional zu einem Paket zusammengefasst.
- **Health Monitoring & Crash-Restart:** Ping/Pong-Überwachung, automatischer Neustart mit Exponential Backoff.
- **Typisierten Transport:** JSON für Skalare, Binär-Chunks für große Daten (NumPy-Arrays, Bytes, Bilder).
- **Persistente Skript-Kontexte:** Skripte und deren Zustand (Importe, Variablen, Funktionen) bleiben in der Instanz erhalten.
- **Hot Reload:** geänderter Python-Code wird erkannt und per Konfiguration neu geladen.
- **Editor-Integration:** Python-Editor-Dock mit Syntax-Highlighting und deterministischer GDScript-Wrapper-Generierung.

2. Architektur



```

└─ BridgeHealthMonitor                                Text = JSON · Binary =
└─ server (asyncio + Worker-Thread)

└─ BridgeProvisioner (venv, pip)                      Header + Chunks
└─ executor (Kontexte, Source-Hash)

└─ introspection (AST)

└─ serializer · protocol (v2)

```

Der Ablauf: Godot startet den Python-Server als Subprozess. Der Server schreibt Port + PID in `python_bridge/tmp/<instanz>.json`. Godot liest die Datei, verbindet sich per WebSocket, sendet ein `hello` und gilt nach der `hello_ack`-Antwort als `READY`.

Invariante: Python wartet nie auf einen Godot-Frame, und Godot wartet nie auf Python. Eingehende Ergebnisse werden im Scheduler gepuffert und innerhalb eines definierten Sync-Punkts (`max_max_results_per_frame` pro Frame) verarbeitet. Jede Anfrage trägt eine eindeutige `id`; Antworten werden darüber den Tasks zugeordnet.

Nachrichten-Typen (Protokoll v2): `hello` → `hello_ack` · `task` / `batch` → `task_result` / `task_error` / `batch_result` · `cancel` → `cancel_ack` · `ping` → `pong` · `reload` → `reload_ack` · `introspect` → `introspect_result` · `shutdown` → `shutdown_ack` · `event` · `status`

3. Installation in einem neuen Projekt

1. Den Ordner `addons/python_bridge` in das Zielprojekt nach `addons/` kopieren.
2. Godot öffnen → Projekt → Projekteinstellungen → Plugins → „Python Bridge“ aktivieren.
Das Plugin registriert den Autoload-Singleton `PythonBridge` automatisch.
3. Python 3.8+ muss installiert sein (die Bridge findet sie über `PATH`, `PYTHON_PATH` oder `python_executable`).
4. Beim ersten Start wird automatisch `python_bridge/venv/` erzeugt und `websockets` (+ konfigurierte Pakete) installiert – das kann 1–3 Minuten dauern.

Hinweis: Das `venv` liegt im Projektordner. Damit Godot es nicht als Ressourcen importiert, enthält

`python_bridge/venv/` eine `.gdignore`-Datei. Das Plugin kopiert die Python-Quellen bei jedem Start nach

`python_bridge/bridge/`, damit Quell- und Laufzeit-Version immer identisch sind.

4. GDScript-API-Referenz

Alle Funktionen sind Instanz-Methoden des Autoload-Singletons `PythonBridge` und dürfen daher `await` verwenden. Aufrufe, die Python ausführen, liefern ein `PythonBridgeResult` (siehe 4.3).

4.1 PythonBridge (Autoload) – Konfiguration & Instanzen

Funktion	Beschreibung
<code>configure(cfg: Dictionary)</code>	Globale Einstellungen setzen (siehe Tabelle in Kapitel 5.1). Alle Schlüssel sind optional; Werte werden typisiert normalisiert.
<code>config() → Dictionary</code>	Aktive Konfiguration zurückgeben (Kopie).
<code>workspace_dir() → String</code>	Aktueller Workspace-Pfad (<code>res://python_bridge</code> als Standard).
<code>start_instance(name := "default") → Result</code>	Startet (bzw. holt) eine Instanz und wartet bis READY (Timeout 180 s). Liefert <code>PythonBridgeResult</code> .
<code>get_instance(name := "default")</code>	Liefert die <code>BridgeInstance</code> (oder null), ohne sie zu starten.
<code>instance_status(name := "default") → String</code>	Status-String: <code>none</code> <code>provisioning</code> <code>starting</code> <code>connecting</code> <code>handshake</code> <code>ready</code> <code>crashed</code> <code>restarting</code> <code>stopping</code> <code>stopped</code> <code>error</code> .
<code>stop_instance(name := "default") · stop_all()</code>	Graceful Shutdown einer/aller Instanzen (sendet shutdown, wartet auf Ack, killt bei Timeout).
<code>shutdown() · shutdown_now()</code>	Blockiert neue Tasks und stoppt alle Instanzen; <code>shutdown_now()</code> erzwingt den sofortigen Stop (keine Zombies).
<code>poll()</code>	Treibt alle Instanzen und den Scheduler pro Frame (automatisch in <code>_process</code> und vom Editor-Plugin).
<code>static system() → Node</code>	Statischer Helfer: findet den Autoload-Node über den Szenenbaum.

Signale: `pulse` (jeden Frame) · `instance_state_changed(instance, state)` · `task_done(task)` (jeder terminale Task) · `bridge_event(instance, event)` (Python → Godot-Events).

4.2 PythonBridge – Skript- & Task-API

Funktion	Beschreibung
<code>execute(code, input := {}, instance := "default", timeout_sec := 30.0)</code>	Führt temporären Python-Code in einem Wegwerf-Kontext aus (Konvention: <code>input / result</code>).
<code>create_script(script_id, code, subfolder := "")</code>	Schreibt ein dauerhaftes Skript nach <code>res://python_bridge/scripts/<subfolder>/<id>.py</code> (überschreibt). Liefert <code>PythonBridgeResult</code> .
<code>get_script_source(script_id)</code> → <code>String</code>	Quelltext des Skripts (leer, wenn nicht vorhanden).
<code>execute_script(script_id, input := {}, instance := "default", timeout_sec := 30.0)</code>	Führt ein dauerhaftes Skript aus (Konvention: <code>input / result</code>).
<code>call_script(script_id, function, args := [], kwargs := {}, instance := "default", timeout_sec := 30.0)</code>	Ruft eine benannte Funktion im Skript-Kontext auf; der Rückgabewert ist das Ergebnis. Der Kontext wird per Source-Hash nur bei Änderung neu definiert.
<code>define_script(script_id, instance := "default", timeout_sec := 30.0)</code>	Lädt ein Skript vorab in den Kontext (ohne Ausführung).
<code>script_path_for(script_id, subfolder)</code> <code>resolve_script_path(script_id)</code>)	Pfad-Helfer; erlaubt volle Pfade oder <code>id / unterordner/id</code> .
<code>submit_task(task: PythonBridgeTask)</code> → <code>Result</code>	Reicht einen Task in den Task Manager ein (Backpressure-Check; Ergebnis kommt über <code>task.done</code>).
<code>cancel_task(task_id)</code> → <code>bool</code>	Bricht einen Task ab (Queue- oder In-Flight).
<code>get_task(task_id)</code> → <code>PythonBridgeTask</code>	Task-Objekt nach ID (für Status/Ergebnis).
<code>hot_reload_script(script_id)</code> → <code>Result</code>	Löst Hot Reload aus – je nach <code>hot_reload_mode</code> : <code>none</code> (deaktiviert), <code>reload_context</code> (Kontext

wird invalidiert; Standard) oder
restart_instance (Instanz neu starten).

```
introspect_script(script_id,  
instance := "default") →  
Result
```

Fragt die Funktions-Signaturen per AST ab (ohne
das Skript auszuführen); Basis für die Wrapper-
Generierung.

4.3 PythonBridgeTask

Ein Task durchläuft die Zustände QUEUED → RUNNING → COMPLETED | FAILED | CANCELLED |
TIMEOUT. Ergebnis und Fehler sind über das Signal done(result: PythonBridgeResult) abrufbar
(genau einmal pro Task).

Feld	Beschreibung
id · instance_id	Eindeutige Task-ID; instance_id = "" bedeutet Auto-Zuordnung an eine beliebige bereite Instanz.
priority	Ganzzahl, 0 = höchste Priorität (stabile Reihenfolge bei gleicher Priorität).
command	run call define
context_id · source · function · args · kwargs · input	Ausführungsdaten des Tasks.
timeout_ms	Maximale Gesamtlebensdauer (Queue + Ausführung); Überschreitung → TIMEOUT.
max_retries · retries_left · retry_policy	Retry-Konfiguration (Policy: none connection_error process_error all).
batchable	Erlaubt dem Scheduler, den Task in ein Batch-Frame zu fassen (Standard true).
result · meta · state	Endergebnis, Metadaten (z. B. duration_ms) und aktueller Zustand.

Konstruktoren (statisch): PythonBridgeTask.make_call(id, context, source, function,
args, kwargs, timeout_ms, priority := 0), make_run(...), make_define(...).

4.4 BridgeInstance

Eine Instanz = ein Python-Subprozess + ein WebSocket-Kanal. Meist reicht die PythonBridge-Facade; für Spezialfälle kann man die Instanz direkt nutzen.

Funktion	Beschreibung
<code>send_message(msg: Dictionary) → Error</code>	Sendet eine Protokoll-Nachricht (nur im READY-Zustand).
<code>wait_ready(timeout := 180) → bool</code>	Wartet (via await auf <code>process_frame</code>) bis READY.
<code>status_text() · is_active() · is_ready() · is_ready_immediately()</code>	Zustandsprüfungen (siehe 4.1 für Status-Strings).
<code>last_error_message() → String</code>	Letzter Fehlertext (Provisioning-/Verbindungsfehler).
<code>stop() · shutdown_now()</code>	Graceful bzw. sofortiger Stop (inkl. Prozess-Kill bei Bedarf).

Signale: `instance_ready · state_changed(instance, state) · message_received(instance, parsed) · lost(instance, error)`. Zustandsmaschine: NONE → PROVISIONING → STARTING → CONNECTING → HANDSHAKE → READY (bei Fehler/Crash: CRASHED → RESTARTING → STARTING mit Backoff, oder ERROR; bei Stop: STOPPING → STOPPED).

4.5 PythonBridgeResult

Feld	Beschreibung
<code>ok / is_ok() / is_error()</code>	Erfolg? Fehler?
<code>status</code>	"ok", "error" (Python-Exception), "timeout", "not_ready", "down" (Prozess getrennt), "internal" (Tool-Problem), "cancelled".
<code>value</code>	Ergebnis bei Erfolg (typisiert, siehe Kapitel 6).
<code>error</code>	Dictionary {code, type, message, traceback, task_id?, instance_id?} bei Fehlern (Taxonomie in 9.2).

<code>error_message()</code> · <code>error_code()</code>	Kurztext bzw. Fehlercode des Fehlers.
<code>request_id</code> · <code>task_id</code> · <code>instance_id</code> · <code>meta</code>	Zuordnungs- und Meta-Daten (z. B. <code>duration_ms</code>).

5. Task Manager & Scheduler

5.1 Konfiguration (PythonBridgeConfig)

Schlüssel	Standard	Bedeutung
<code>autostart</code>	<code>false</code>	Instanz(en) beim Plugin-Start automatisch hochziehen.
<code>workspace_dir</code>	<code>res://python_bridge</code>	Workspace (<code>scripts/</code> , <code>tmp/</code> , <code>config/</code> , <code>venv/</code> , <code>bridge/</code>).
<code>python_executable</code>	<code>""</code>	Leer = automatische Python-Erkennung.
<code>dependencies</code>	<code>[]</code>	Zusätzliche pip-Pakete (z. B. <code>["numpy"]</code>).
<code>max_queued_tasks</code>	<code>1000</code>	Backpressure: maximale Queue-Länge.
<code>max_inflight_per_instance</code>	<code>1</code>	Maximale gleichzeitig laufende Einheiten pro Instanz.
<code>max_payload_bytes</code>	<code>64 MiB</code>	Maximale Task-Payload (Quelle + Input/Args).
<code>task_timeout_ms</code>	<code>30000</code>	Standard-Timeout für Tasks.
<code>max_retries</code> · <code>retry_policy</code> · <code>retry_delay_ms</code>	<code>0</code> · <code>connection_error</code> · <code>250</code>	Retry-Konfiguration des Task Managers.
<code>max_batch_size</code> · <code>max_batch_delay_ms</code>	<code>32</code> · <code>32</code>	Batch-Fenster: Größe und

		maximale Wartezeit.
max_dispatch_per_frame · max_results_per_frame · max_inbox_size	16 · 64 · 512	Frame-Budgets des Schedulers; volle Inbox throttelt den Dispatch.
connect_timeout_ms · provision_venv_timeout_ ms · provision_pip_timeout_m s · shutdown_timeout_ms	20000 · 120000 · 300000 · 3000	Zeitlimits für Verbindung, Provisioning und Shutdown.
health_check_interval_m s · health_missed_pong_limi t	5000 · 3	Health Monitoring: Ping- Kadenz und verpasste Pongs bis zum Crash.
max_restart_attempts · restart_base_delay_ms · restart_backoff_factor · stable_uptime_ms	3 · 500 · 2 · 30000	Crash-Restart: Versuche, Backoff (500 → 1000 → 2000 ms), Reset nach stabiler Laufzeit.
hot_reload_mode	"reload_cont ext"	none reload_context restart_instance.
auto_generate_wrappers · wrapper_dir	false · res://python _bridge/wrap pers	Automatische Wrapper- Generierung (Standard: explizit) und Zielordner.

5.2 Verhalten

- **Priorität:** 0 = höchste; stabile Reihenfolge (FIFO) bei gleicher Priorität.
- **Timeout:** ein Timer pro Task (Queue + Ausführung); bei Überschreitung wird ein CANCEL best-effort gesendet und der Task auf TIMEOUT gesetzt.
- **Cancel:** Queue-Tasks werden sofort entfernt; laufende Tasks werden markiert und als CANCELLED abgeschlossen (späte Ergebnisse werden verworfen).
- **Retry:** nur gemäß retry_policy; retries_left wird dekrementiert, der Task mit retry_delay_ms neu eingereiht.
- **Backpressure:** max_queued_tasks und max_payload_bytes lehnen Tasks mit strukturiertem Fehler ab (Queue voll / Payload zu groß).
- **Frame-Sync:** Ergebnisse landen in einer begrenzten Inbox; pro Frame werden max. max_results_per_frame verarbeitet. Ein wachsender Backlog throttle den Dispatch – stabile Framezeiten statt maximaler Antwortgeschwindigkeit.
- **Batching:** kompatible, batchbare Tasks derselben Priorität an dieselbe Instanz werden bis

max_batch_size oder max_batch_delay_ms gesammelt. Ein einzelner Task wartet nie auf weitere; eine höhere Priorität bricht das Fenster ab. Reihenfolge und Semantik bleiben erhalten; ein Fehler innerhalb eines Batches betrifft nur den betroffenen Item (separate Ergebnisse).

- **Crash-Behandlung:** bei Verbindungsabbruch oder Prozess-Ende werden In-Flight-Tasks gemäß Policy gefailt oder erneut eingereicht; die Instanz startet mit Exponential Backoff neu (Counter-Reset nach stable_uptime_ms).

6. Datentransport & Typen

Skalare bleiben lesbares JSON. Strukturierte Werte werden als getaggte Objekte {"\$pb": "<tag>", ...} kodiert. Große Binär-Blobs (> 512 Bytes) wandern nicht als Base64 in den JSON-Header, sondern als Binär-Chunks im Frame (U32LE(Header-Länge) + HeaderJSON + [U32LE(Chunk-Länge) + Chunk]*). Die Typ-Mapping-Tabelle ist zentral in PythonBridgeTypeMapper und um benutzerdefinierte Typen erweiterbar (register(tag, encode_fn, decode_fn, custom_class := "")).

Python → Godot

Godot-Typ

None / bool / int
/ float / str

null / bool / int / float / String

list / tuple /
set

Array

dict

Dictionary (Schlüssel müssen str/int/float/bool sein)

bytes / bytearray

PackedByteArray

numpy.ndarray
(diverse dtypes)

PackedFloat64Array / PackedFloat32Array /
PackedInt32Array / PackedInt64Array / PackedByteArray
(je nach dtype)

numpy.integer /
numpy.floating

int / float

beliebiges anderes
Objekt

String (repr-Fallback, Tag pyobject)

Godot → Python

Python-Typ

Vector2/3/4 · Color ·
Transform3D

list der Komponenten (z. B. [x, y,
z])

PackedByteArray (groß)	bytes (als Chunk)
PackedInt/Float-Arrays	bytes (roh, für <code>numpy.frombuffer</code>)
Array / Dictionary	list / dict
Image	rohe Pixel-Daten + Metadaten (w, h, format)

Endianness: Längen-Präfixe im Binärformat sind Little-Endian auf beiden Seiten (`struct.pack("<I",`

`...)` in Python, `decode_u32` / explizite Byte-Kodierung in Godot).

7. Python-Seite

7.1 Skript-Konventionen

- `execute` / `execute_script`: Das Skript erhält die Eingabe in der Variablen `input` und legt das Ergebnis in `result` ab.
- `call_script`: Die benannte Funktion wird mit `args` / `kwargs` aufgerufen; ihr Rückgabewert geht an Godot.
- **Persistente Kontexte:** Jede `context_id` hat einen eigenen Namespace. Der Executor speichert einen Source-Hash; `call_script` re-definiert den Kontext nur, wenn sich der Quelltext geändert hat.
- **Fehler:** Python-Exceptions werden strukturiert gemeldet (`{code, type, message, traceback}`) – der Server-Prozess bleibt am Leben.
- **stdout/stderr:** werden pro Task erfasst und im Ergebnis mitgeliefert.

7.2 Aufbau des Python-Pakets

Datei	Zweck
<code>run_server.py</code>	Entry-Point: CLI-Parameter (<code>--bind --port --tmpdir --tag</code>), startet den asyncio-Server.
<code>python_bridge</code> <code>/server.py</code>	WebSocket-Server: routet <code>task</code> / <code>batch</code> / <code>cancel</code> / <code>reload</code> / <code>introspect</code> / <code>ping</code> / <code>hello</code> / <code>shutdown</code> ; Nutzer-Code läuft in genau einem Worker-Thread pro Instanz, damit der Event-Loop reaktionsfähig bleibt; Watchdog beendet den Prozess nach Shutdown/Verbindungsabbruch.
<code>python_bridge</code>	Verwaltet Kontexte, führt <code>define</code> / <code>run</code> / <code>call</code> aus (Source-Hash),

<code>/executor.py</code>	fängt Exceptions, capturet stdout/stderr, unterstützt Cancellation.
<code>python_bridge / introspection .py</code>	AST-basierte Funktions-Signatur-Analyse (führt nie Code aus) – Grundlage der Wrapper-Generierung.
<code>python_bridge /protocol.py</code>	v2-Nachrichtentypen und Batch-Item-Dekodierung.
<code>python_bridge / serializer.py</code>	<code>encode_obj</code> / <code>decode_obj</code> : typisierte Serialisierung inkl. NumPy-Arrays und Chunks.
<code>requirements. txt</code>	Minimale Abhängigkeit: <code>websockets>=11</code> (weitere Pakete pro Projekt konfigurierbar).

8. Editor-Integration

Das Plugin registriert den Autoload PythonBridge und hängt ein leichtgewichtiges Dock-Panel (rechts oben) an. Der Editor spricht ausschließlich mit der Facade – keinerlei Runtime-Logik im Editor.

Element	Beschreibung
Dock-Panel (<code>editor/python_editor.gd</code>)	Skriptliste (auto-refresh), CodeEdit mit Python-Syntax-Highlighting, Aktionen: Save · Run · Generate wrapper · Hot reload · Refresh · New script , Status + Log, mtime-Watcher für externe Änderungen.
Syntax-Highlighting (<code>editor/python_syntax_highlighter.gd</code>)	Keywords, Builtins, Strings, Kommentare, Zahlen, Dekoratoren.
Wrapper-Generator (<code>editor/wrapper_generator.gd</code>)	Deterministisch (gleiches Schema → identische Bytes); Marker-Header; überschreibt nur markierte Dateien; <code>class_name</code> -Kollisions-Check; Python-Defaults bleiben autoritativ (Sentinel-Check).

Wrapper-Beispiel: Aus `def calculate(a: int, b: int) -> int: return a + b` erzeugt der Generator ein GDScript wie `bridge.calculate(2, 3)`. Der originale `.py`-Code bleibt unverändert und außerhalb von Godot lauffähig. Wrapper werden nur erzeugt, wenn man sie anfordert (Button bzw. `auto_generate_wrappers`).

9. Demo-Szene & Beispiele

Das Repository enthält eine fertige Demo-Szene (`example/demo_scene.tscn`) mit Nodes und angehängten Skripten – öffnen und **F6** (Aktuelle Szene ausführen) drücken. Der Button **Bridge starten** startet die Python-Runtime, danach kann man die einzelnen Aspekte per Button ausprobieren. Jedes Demo-Node ist ein eigenes Skript unter `example/demo/`:

Node / Skript	Demonstriert
DemoRoot – <code>demo_root.gd</code>	Orchestriert Bridge-Start, Workspace-Sync (<code>example/scripts/*.py</code> → <code>python_bridge/scripts/</code>), Shutdown.
DemoBasic – <code>demo_basic.gd</code>	<code>call_script</code> (Args/Kwargs), <code>execute_script</code> (input/result), <code>define_script</code> – inkl. strukturierter Typen (Array, Dictionary).
DemoTasks – <code>demo_tasks.gd</code>	Low-Level-Task-API: <code>PythonBridgeTask.make_call</code> , <code>submit_task</code> , done-Signal, Priorität, Timeout, <code>cancel_task</code> .
DemoBatch – <code>demo_batch.gd</code>	8 schnelle Aufrufe → Task-Batching (<code>max_batch_size/max_batch_delay_ms</code>) mit erhaltener Reihenfolge.
DemoErrors – <code>demo_errors.gd</code>	Strukturierte Fehler: Python-Exception (Typ/Message/Traceback/Task-ID), Timeout, unbekanntes Skript.
DemoMulti – <code>demo_multi.gd</code>	Zweite Python-Instanz (worker) parallel zur Standard-Instanz, Task- Verteilung, Statusabfrage, Stop.

Die Beispiel-Python-Skripte liegen in `example/scripts/`: `demo_skript.py` (`calculate`, `greet`, `fibonacci`, `vector_list`, `boom`, `slow`, `big_blob`) und `crash_skript.py` (`crash()` beendet den Python-Prozess absichtlich – zeigt Crash-Erkennung und Exponential-Backoff-Restart der Bridge).

10. Lebenszyklus, Provisioning & Fehlerbehandlung

9.1 Lebenszyklus & Provisioning

1. **PROVISIONING** – Workspace-Ordner anlegen (`scripts/`, `tmp/`, `config/`), Python erkennen (≥ 3.8), Bridge-Python nach `python_bridge/bridge/` kopieren.
2. **venv** – falls nicht vorhanden: `python -m venv ../venv` (nicht-blockierend, Timeout 120 s).
3. **pip** – `pip install websockets>=11` + konfigurierte Dependencies (Timeout 300 s, Log in `tmp/pip.log`), anschließend Import-Verifikation.
4. **STARTING** – `run_server.py` als Subprozess starten; Server schreibt `tmp/<instanz>.json` (Port + PID).

5. **CONNECTING** → **HANDSHAKE** – WebSocket-Verbindung aufbauen, hello senden.
6. **READY** – hello_ack empfangen; Tasks sind jetzt möglich.

Crash & Restart: Ein unerwarteter WebSocket-Close oder ein toter Subprozess im READY-Zustand löst die Restart-Policy aus: max_restart_attempts Versuche mit Exponential Backoff (500 ms → 1000 ms → 2000 ms, konfigurierbar), Counter-Reset nach stable_uptime_ms. Es bleiben keine Zombie-Prozesse zurück (vor jedem Neustart wird ein eventuell noch laufender Prozess beendet).

Graceful Shutdown: Beim Beenden von Godot (shutdown()/_exit_tree) werden neue Tasks blockiert, shutdown an alle Instanzen gesendet, auf shutdown_ack (bzw. Timeout shutdown_timeout_ms) gewartet und Prozesse notfalls zwangsweise beendet.

9.2 Fehler-Taxonomie

Jeder Fehler trägt einen Code aus der Taxonomie (PythonBridgeErrorHandler). Der Status eines PythonBridgeResult wird daraus abgeleitet.

Code	Bedeutung	Status
PYTHON_EXCEPTION · TASK_ERROR	Python-Exception bzw. Task-Fehler	error
TIMEOUT_ERROR	Antwort kam nicht rechtzeitig	timeout
PROCESS_ERROR · CONNECTION_ERROR	Prozess-/ Verbindungsfehler	down
DEPENDENCY_ERROR · SERIALIZATION_ERROR · PROTOCOL_ERROR · BRIDGE_ERROR	Infrastrukturfehler	internal

Eine Python-Exception enthält mindestens: Exception-Typ, Message, Traceback, Task-ID, Instanz-ID – nie nur „Something went wrong“.

11. Troubleshooting

Problem / Status	Ursache & Lösung
status = "not_ready"	Instanz wurde nicht gestartet oder noch nicht READY – vor Aufrufen await start_instance() nutzen bzw. wait_ready().

<code>status = "timeout"</code>	Antwort kam nicht in <code>timeout_sec</code> (Standard 30 s). Timeout erhöhen oder Python-Code beschleunigen.
<code>status = "error"</code>	Python-Exception im Nutzer-Code – Details in <code>result.error</code> (type / message / traceback).
<code>status = "down"</code>	Python-Prozess getrennt (WebSocket geschlossen) – z. B. Absturz im Server oder <code>stop()</code> . Prozess-Log prüfen; die Instanz restartet gemäß Backoff-Konfiguration.
<code>status = "cancelled"</code>	Task wurde vor Abschluss abgebrochen (<code>cancel_task</code> oder Shutdown).
„Kein Python gefunden“	<code>python_executable</code> setzen, <code>PYTHON_PATH</code> definieren oder Python zum PATH hinzufügen.
venv/pip hängt	Timeout 120/300 s; Details in <code>python_bridge/tmp/pip.log</code> . Ggf. venv-Ordner löschen und neu provisionieren lassen.
„Python-Prozess getrennt“ beim ersten Aufruf	Meist ein Server-/Protokollproblem beim Binär-Versand – Version von <code>addons/python_bridge/python</code> (Quelle) und <code>python_bridge/bridge</code> (Laufzeit-Kopie) muss identisch sein; das Plugin kopiert bei jedem Start automatisch.
Werte kommen als String an	Objekt war nicht serialisierbar → <code>pyobject</code> -Fallback (<code>repr</code>). Nur JSON-fähige Typen, <code>bytes</code> , <code>ndarray</code> , Listen/Dicts verwenden.
Compiler-Fehler „Could not resolve class ... parser error“	Alter <code>.godot</code> -Cache aus einer früheren Addon-Version. Projekt einmal schließen, <code>.godot/</code> löschen und erneut öffnen (die <code>.uid</code> -Dateien sind im Repo enthalten).

Python Bridge v0.2.1 – dokumentiert nach dem Quellcode in `addons/python_bridge/` (Godot-Seite: `core/`, `editor/`; Python-Seite: `python/`). Weitere Details: `docs/ARCHITEKTUR.md`, `docs/API.md`, `docs/INSTALLATION.md`, `docs/TROUBLESHOOTING.md`, `docs/CHANGELOG.md`.